

Justin Christopher

Enhancement One: Software Design/Engineering Narrative

The artifact I selected for this enhancement is my Android Inventory App I originally developed in CS 360. The app was built to track inventory levels and demonstrate CRUD functions. It features a login screen where users can login or create an account and an inventory screen where items, quantities, and low-stock thresholds are managed. Rows highlight red when an item falls below its threshold.

Justification

I selected this artifact because it represents a realistic full stack mobile application that touches several eras of computer science including database design, user interface development, and security. It is something I also built from scratch, so I had a decent understanding of its architecture (after a refresh read through of its code. It had been a while since I looked at it.)

The specific enhancement I made for this category was implementing a role-based access control system. In the original version of the application, any user who created an account got instant access to the inventory with Read and Write permissions. This is a significant security gap in any real business environment. My enhancement added three layers of control. First, New user accounts are created with a pending status. While their account has this status they cannot log in. An admin will have to log in and explicitly approve their access for them to sign on. Second, users assigned a role of either admin or regular user. Third, regular users can be granted either read-only access or read-write access. The admin has complete control on who can add, edit, and delete inventory items.

To support this, I modified the SQLite database schema to add roles, account status, and

edit permission columns to the users table. I created a new AdminActivity with its own XML layout and RecyclerView-based user list that lets the admin approve, deny, and toggle permissions for each account. I updated LoginActivity to check account status before allowing login and to route admin users to the management screen rather than the inventory screen. I updated InventoryActivity to read the logged in username from the Intent and hide or block write actions for read-only users. A default admin account is seeded automatically on the first install.

Course Outcomes

I planned on meeting course outcomes of two and five with this enhancement and I believe I met both. For outcome five, developing a security mindset, the enhancement directly addresses a real access control vulnerability in the original design. Unauthenticated and unapproved users can no longer login to the application and access sensitive data. The principle of least privilege is now enforced so users only receive the permissions they need. For outcome two, professional quality communication, the code includes clear comments explaining design decisions, constant names that are descriptive, and the logic flow is organized so another developer could read and maintain it if needed.

I do not have any updates to my coverage plan. The enhancement went as described in module one.

Reflection

The most important thing I learned during this enhancement was how much a small architectural decision early in development can create security problems later. The original app was built with the assumption that any registered user was a trusted user. Adding access control after the fact required touching nearly every layer of the application including the database schema, the login flow, the inventory screen, and a brand-new activity. If this had been designed with access

control in mind from the start, it would have been simpler to implement than rework existing code. The biggest problem I had was a bug in my schema when I was trying to test the changes. I missed some spaces, so the schema was silently failing and not creating the seeded admin account.